КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет інформаційних технологій
Кафедра програмних систем і технологій

ЗВІТ

з лабораторної роботи №4
з дисципліни «Об'єктно-орієнтоване програмування»

Тема: «Застосування об'єктно-орієнтованого підходу при створенні графічних інтерфейсів користувача»

Проєкт: «Автоматизована система “Склад запчастин для авто”»

Виконав: студент групи ІПЗ-21, підгрупа 1
Агапов Олександр Олексійович

Перевірила: доц. Зубик Л. В.

Київ - 2026

РЕФЕРАТ

Звіт про лабораторну роботу: 22 с., 6 табл., 17 рис., 7 джерел.

Ключові слова: ГРАФІЧНИЙ ІНТЕРФЕЙС, C#, ASP.NET CORE MVC, SQLITE, ENTITY FRAMEWORK CORE, IDENTITY, CRUD, СКЛАД, ВИДАЧА, ПОВЕРНЕННЯ.

Об’єкт розроблення – вебзастосунок автоматизованої системи обліку автозапчастин і його інтерфейс для керування даними складу.

Мета роботи – застосувати об’єктно-орієнтований підхід до створення графічного інтерфейсу та забезпечити роботу із записами бази даних проєкту попередніх лабораторних робіт.

Методи розроблення: мова C#, платформа ASP.NET Core MVC, подання Razor, Entity Framework Core, SQLite та ASP.NET Core Identity. Для перевірки використано запуск вебзастосунку, HTTP-запити, браузерні дії та контрольні запити до ізольованої бази SQLite.

Результати: реалізовано керування каталогом, постачальниками, моделями автомобілів та замовленнями; розмежовано ролі адміністратора, менеджера і комірника; оформлено видачу, підтвердження оплати, повернення та інвентаризаційне коригування. Для складських операцій зберігається історія.

ЗМІСТ

ВСТУП.....4
1 АРХІТЕКТУРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....4
2 ГРАФІЧНИЙ ІНТЕРФЕЙС І РОЛІ.....5
3 ОПЕРАЦІЇ CRUD ДЛЯ ТАБЛИЦЬ БАЗИ ДАНИХ.....8
4 ПЕРЕВІРКА ВВЕДЕННЯ ТА ЦІЛІСНОСТІ.....13
5 БІЗНЕС-ЛОГІКА СКЛАДСЬКИХ ОПЕРАЦІЙ.....14
6 ПЕРЕВІРКА РОБОТИ ЗАСТОСУНКУ.....20
ВИСНОВКИ.....21
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ.....22

ВСТУП

Мета лабораторної роботи – опанувати розроблення графічного інтерфейсу користувача із застосуванням об’єктно-орієнтованого підходу. Для реалізованого раніше проєкту потрібно надати користувачеві інтерфейс на C# та забезпечити додавання, коригування і вилучення записів у таблицях бази даних з урахуванням їхніх зв’язків.

Предметна область проєкту – склад запчастин для автомобілів. Система зберігає каталог, постачальників, сумісність із моделями автомобілів, замовлення, видачі та інвентаризаційні коригування. Графічний інтерфейс відкривається в браузері; C#-контролери обробляють запити, а Razor формує HTML-сторінки [1].

У звіті розглянуто архітектуру, права ролей, операції для кожної сутності, перевірку даних і складські процеси. Результати перевірок отримано на окремій тестовій базі; для демонстраційних облікових записів паролі у звіті не наводяться.

1 АРХІТЕКТУРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Застосунок побудовано за схемою Model–View–Controller. Моделі описують предметні дані; контролери приймають HTTP-запити й виконують правила; Razor Views відображають таблиці та форми. ViewModel переносить дані конкретної форми, коли безпосереднє редагування сутності є незручним або небезпечним [1].

WarehouseContext успадковує IdentityDbContext<ApplicationUser> і містить набори DbSet предметних сутностей. Entity Framework Core відображає об’єкти на таблиці SQLite, а міграції описують зміну схеми [2, 3]. Навігаційні властивості задають зв’язки, зокрема один постачальник – багато замовлень і багато запчастин – багато моделей автомобілів.

Таблиця 1 – Основні класи та їхнє призначення

Клас	**Призначення**
AutoPart, Supplier, VehicleModel	Каталог, постачальник і модель авто; зв’язок сумісності many-to-many.
PurchaseOrder, PurchaseOrderItem	Замовлення постачальнику та його позиції.
PartIssue, StockAdjustment	Документ видачі з переходами стану та журнал коригувань.
ApplicationUser, WarehouseContext	Обліковий запис Identity та контекст доступу до SQLite.
AutoPartsController	Каталог, видачі, повернення та інвентаризаційні коригування.
PurchaseOrdersController, SuppliersController, VehicleModelsController	Замовлення й довідники; перевірка станів і зв’язків.

Клас	Призначення
ReportsController, AccountController, AdminController	LINQ-звіти, вхід та керування користувачами.
IssuePartViewModel та моделі форм	Передача введених даних і правил перевірки для конкретної сторінки.

ООП проявляється в інкапсуляції стану об’єктів і правил роботи з ними, композиції замовлення з позицій та наслідуванні контексту від IdentityDbContext. Узагальнені DbSet<T> і колекції ICollection<T> використовуються для типізованої роботи із сутностями. Кожна дія контролера повертає відповідне подання або перенаправлення.

2 ГРАФІЧНИЙ ІНТЕРФЕЙС І РОЛІ

Користувач входить через сторінку Account/Login. ASP.NET Core Identity перевіряє пароль; після входу авторизація на сервері перевіряє роль для кожної дії. Недоступні пункти меню та кнопки приховані, але вирішальною є серверна перевірка [4].

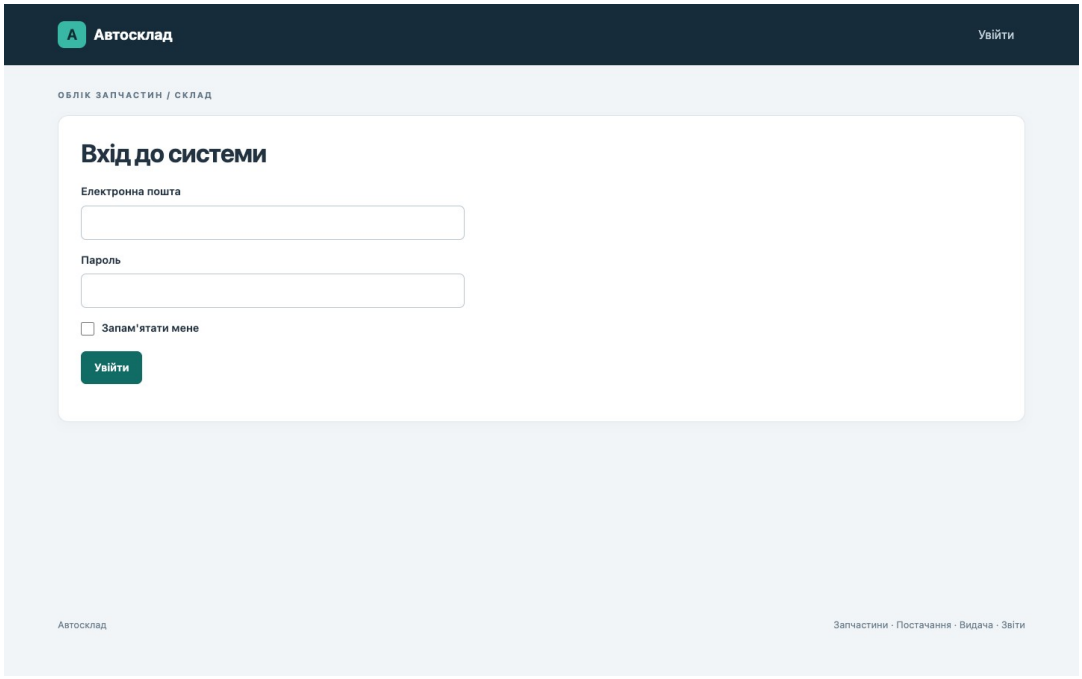


Рисунок 1 – Сторінка входу до системи

Administrator керує обліковими записами: створює їх, призначає одну з трьох ролей і блокує або розблоковує користувачів. Власну роль і блокування через ці дії адміністратор змінити не може. У демонстраційному режимі початкові паролі задаються змінними середовища та не зберігаються в коді проєкту.

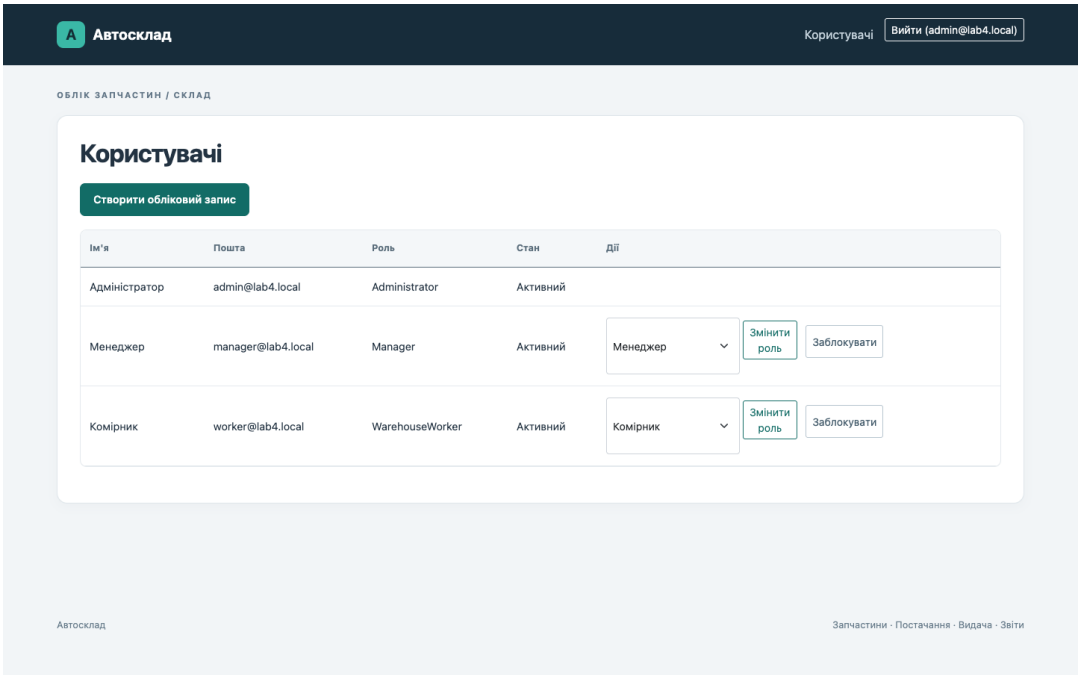


Рисунок 2 – Адміністративне керування користувачами

Таблиця 2 – Ролі та дозволені операції

Операція	**Administrator**	**Manager**	**WarehouseWorker**
Користувачі та ролі	так	ні	ні
Перегляд каталогу і сумісності	ні	так	так
CRUD довідників	ні	так	ні
Створення та зміна замовлень	ні	так	ні
Приймання замовлення	ні	ні	так
Підтвердження оплати	ні	так	ні
Оформлення, видача, повернення	ні	перегляд	так
Коригування запасів	ні	перегляд	так
Повні LINQ-звіти	ні	так	ні

Менеджер працює з каталогом, постачальниками, моделями автомобілів і замовленнями, бачить історію та звіти. Комірник бачить каталог і залишки, приймає замовлення та виконує складські операції. Сторінки каталогу для різних ролей показано на рисунках 3 і 4.

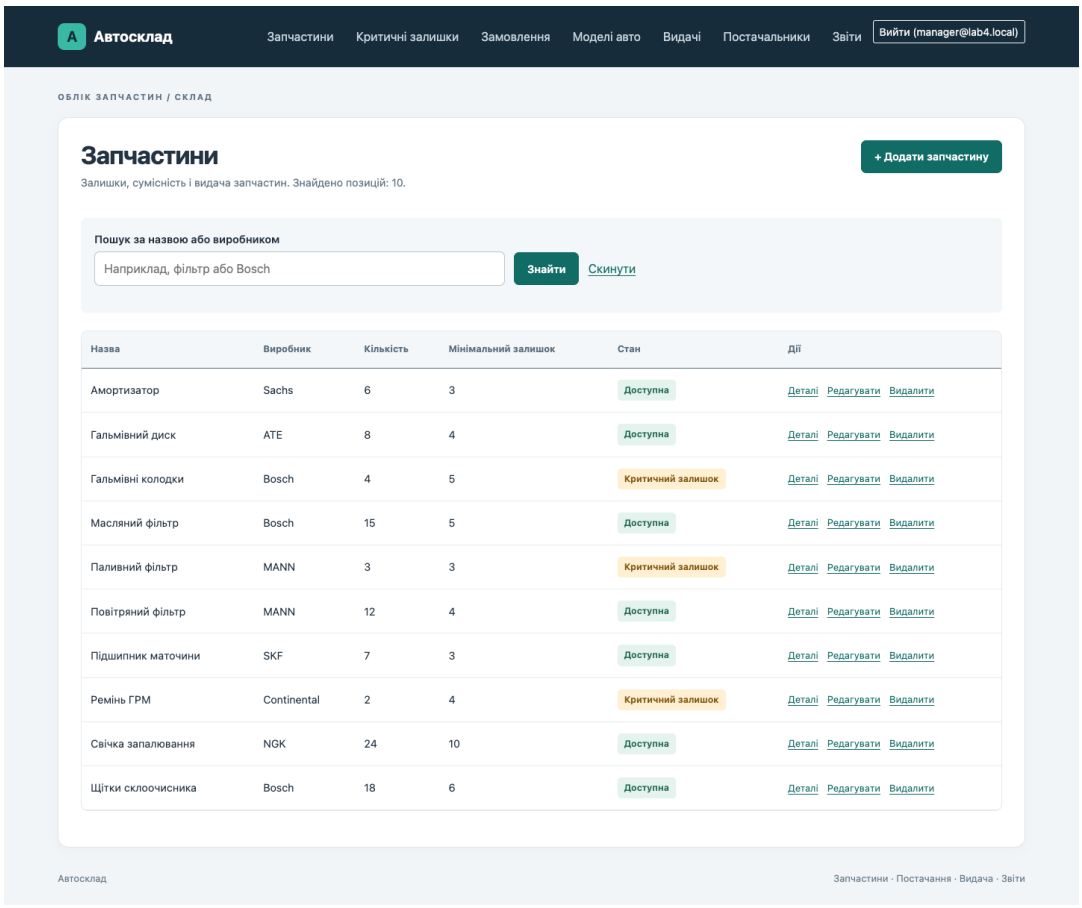


Рисунок 3 – Каталог запчастин у ролі менеджера

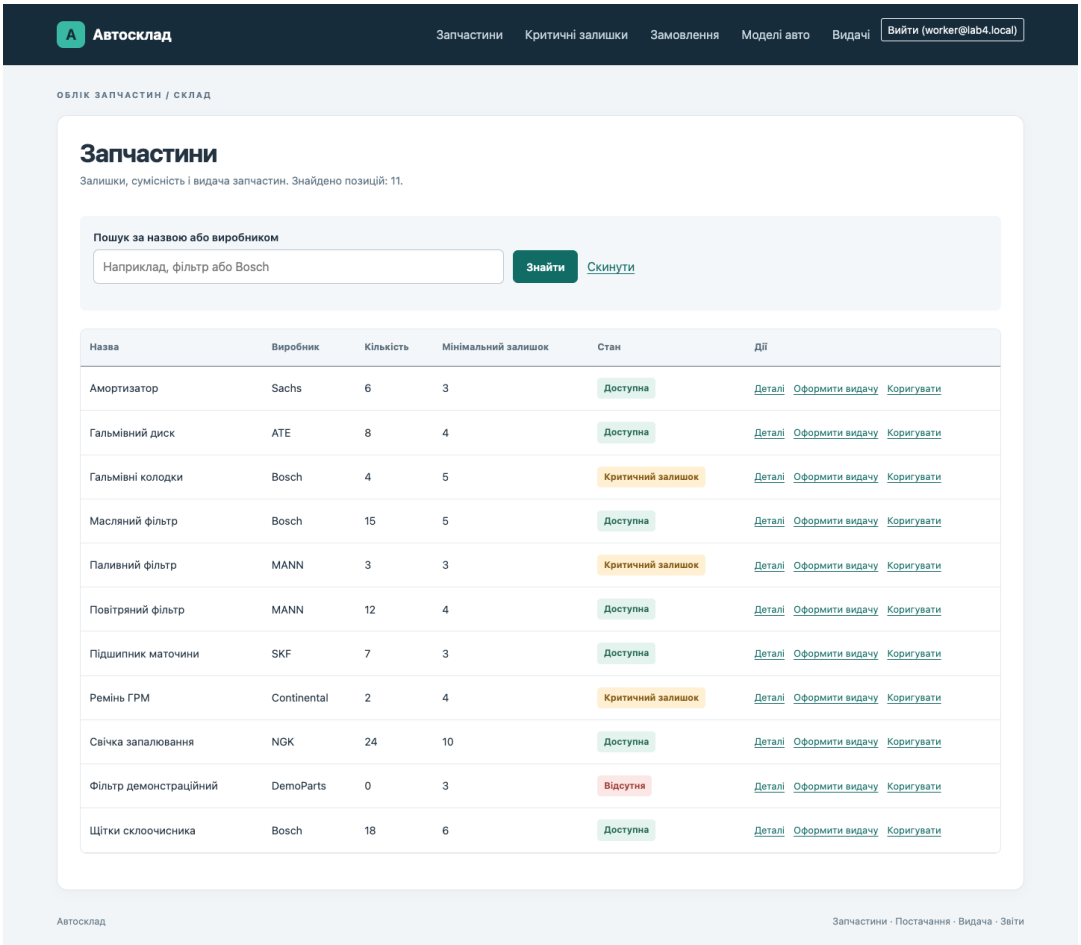


Рисунок 4 – Каталог запчастин у ролі комірника

3 ОПЕРАЦІЇ CRUD ДЛЯ ТАБЛИЦЬ БАЗИ ДАНИХ

CRUD означає створення, перегляд, зміну та вилучення даних. У довідниках ці дії виконуються безпосередньо через форми. Для операційних записів зміна стану або заборона видалення зберігає історію й запобігає повторній зміні залишку. Узагальнення наведено в таблиці 3.

Таблиця 3 – Операції для сутностей

Сутність	**Create / Read**	**Update**	**Delete або обмеження**
AutoPart	Create, Index, Details	Edit: назва, виробник, мінімум	DeleteConfirmed, якщо немає історії
Supplier	Create, Index	Edit	DeleteConfirmed, якщо немає замовлень
VehicleModel	Create, Index	Edit і сумісність	DeleteConfirmed; видаляються зв’язки
PurchaseOrder	Create, Index, Details	Edit до отримання	DeleteConfirmed змінює стан на Cancelled
PurchaseOrderItem	Через форму замовлення	Заміна позицій до отримання	Вилучення з форми до отримання
PartIssue	Issue, Issues, Details	ConfirmPayment, CompleteIssue, ReturnIssue	CancelIssue лише для Pending
StockAdjustment	AdjustStock, Details	Не редагується	Не видаляється через GUI

Сутність	Create / Read	Update	Delete або обмеження
ApplicationUser	Admin/Create, Admin/Index	ChangeRole, SetLocked	Фізичне видалення не передбачене

3.1 Запчастини AutoParts

AutoPart містить Id, Name, Manufacturer, QuantityInStock і MinimumStock. Він пов’язаний з позиціями замовлень, видачами, коригуваннями та моделями авто. Менеджер відкриває Index, Create, Edit і Delete; Details доступна також комірнику. Нова запчастина створюється з нульовим залишком. Edit змінює характеристики, але не приймає QuantityInStock із форми.

DeleteConfirmed перевіряє наявність видач, позицій замовлень і коригувань. Запис з історією вилучити не можна. Під час перевірки тестовій запчастині було змінено назву і мінімум; підроблене поле кількості 999 у POST не змінило залишок. Форма редагування показана на рисунку 5.

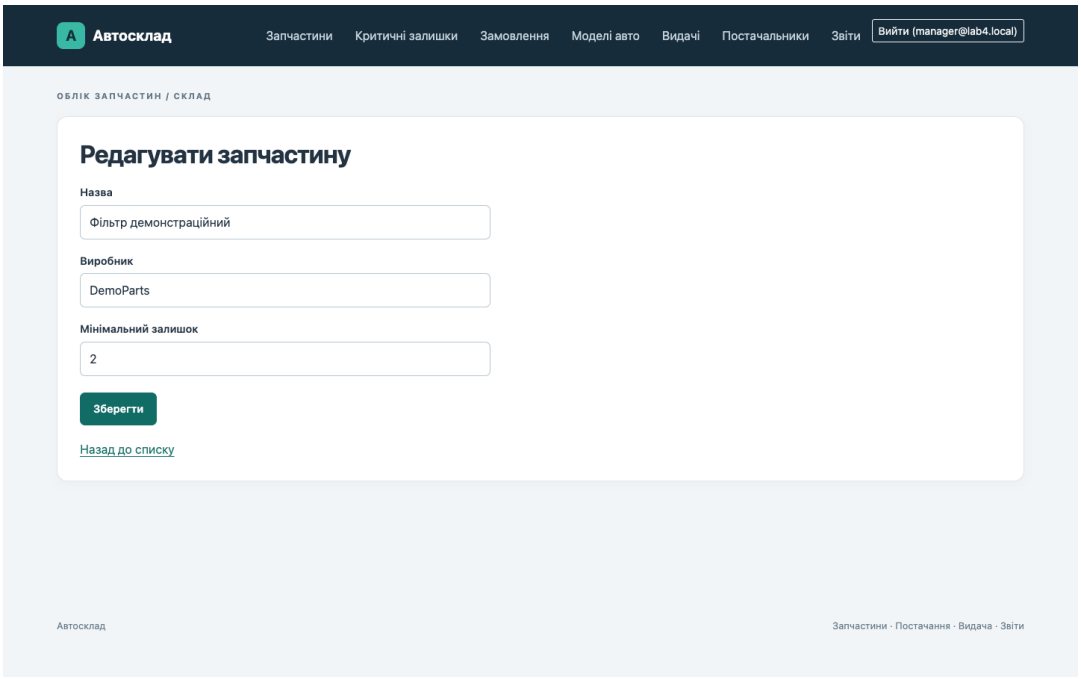


Рисунок 5 – Редагування характеристик запчастини без поля залишку

3.2 Постачальники Suppliers

Supplier містить Id, Name і Phone та має колекцію PurchaseOrders. SuppliersController надає менеджеру Index, Create, Edit і Delete. Create і Edit перевіряють модель; DeleteConfirmed перед видаленням перевіряє пов’язані замовлення. Через GUI користувач відкриває таблицю постачальників, обирає дію та підтверджує видалення на окремій сторінці.

На тестовій базі постачальника без замовлень було створено, змінено і видалено. Спроба видалити AutoTech Supply з пов’язаним замовленням залишила запис у базі й показала повідомлення про неможливість видалення (рисунки 6 і 7).

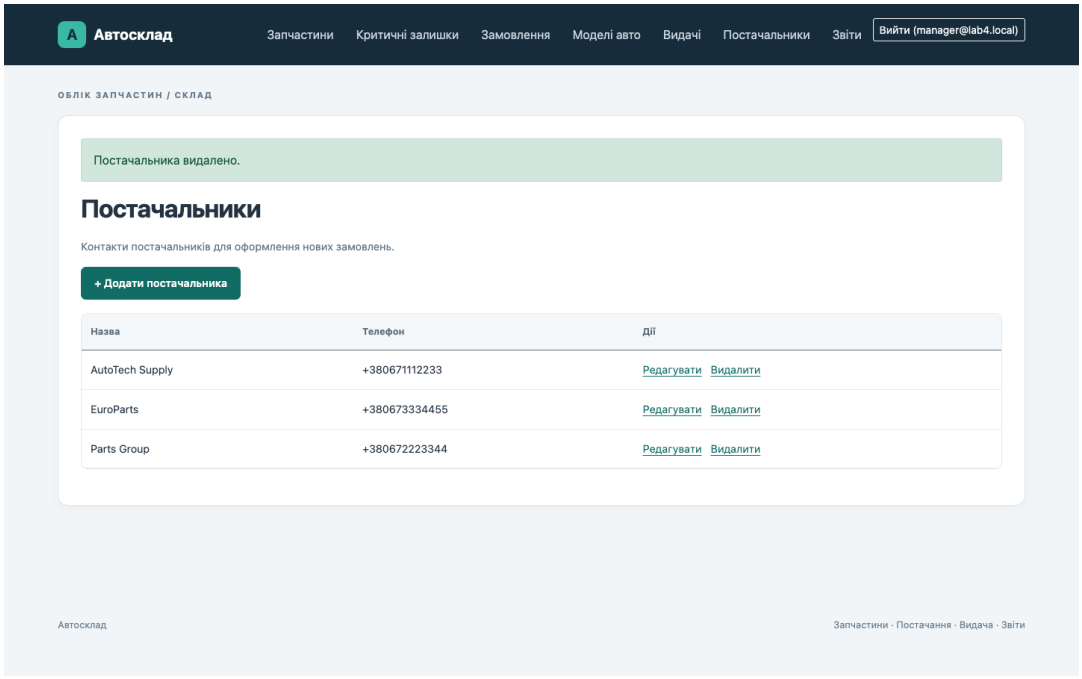


Рисунок 6 – Результат дозволеного видалення постачальника

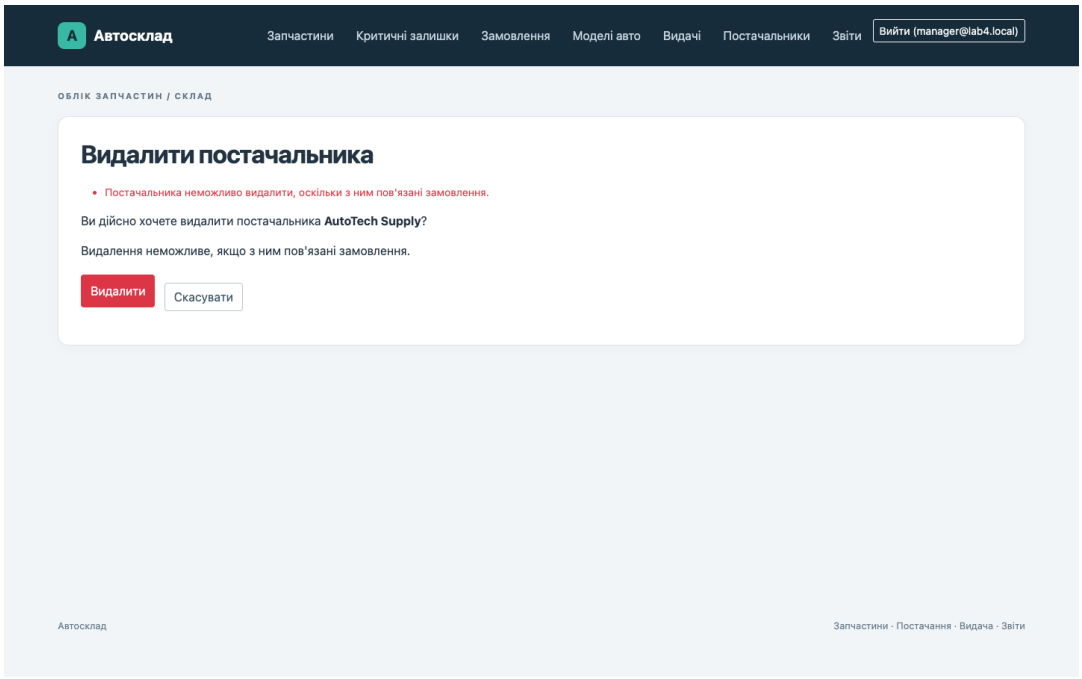


Рисунок 7 – Заборона видалення постачальника із замовленнями

3.3 Моделі автомобілів VehicleModels

VehicleModel зберігає Id, Brand, ModelName і ProductionYear. Зв’язок з AutoPart є many-to-many; проміжна таблиця зберігає лише ключі. VehicleModelsController реалізує Index, Create, Edit і DeleteConfirmed. У

формі VehicleModelViewModel передає SelectedAutoPartIds; під час зміни контролер зберігає вибраний набір сумісних запчастин.

У перевірці створено модель з двома зв’язками, потім залишено один. Після видалення моделі сама запчастина збереглася. Сторінку таблиці моделей показано на рисунку 8.

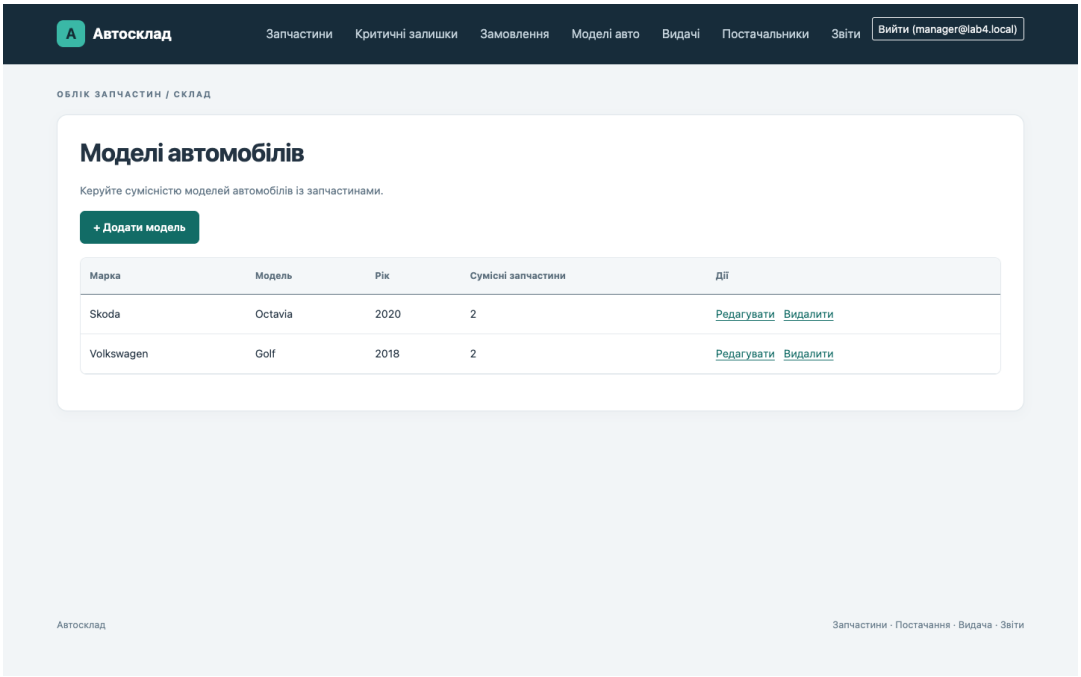


Рисунок 8 – Сторінка моделей автомобілів і дії менеджера

3.4 Замовлення PurchaseOrders

PurchaseOrder містить Id, OrderDate, Status, SupplierId і колекцію Items. PurchaseOrdersController надає Index, Details, Create та Edit. Менеджер створює або редагує неотримане замовлення, а комірник через Receive приймає поставку. Створення замовлення не змінює складський залишок.

Для неотриманого замовлення дія DeleteConfirmed зберігає запис, встановлює стан Cancelled, причину, час та ідентифікатор користувача. Отримане або анульоване замовлення не можна змінити й повторно прийняти. Під час тесту замовлення з позиціями 4 і 2 одиниці було прийнято один раз: залишок змінився з 0 до 6.

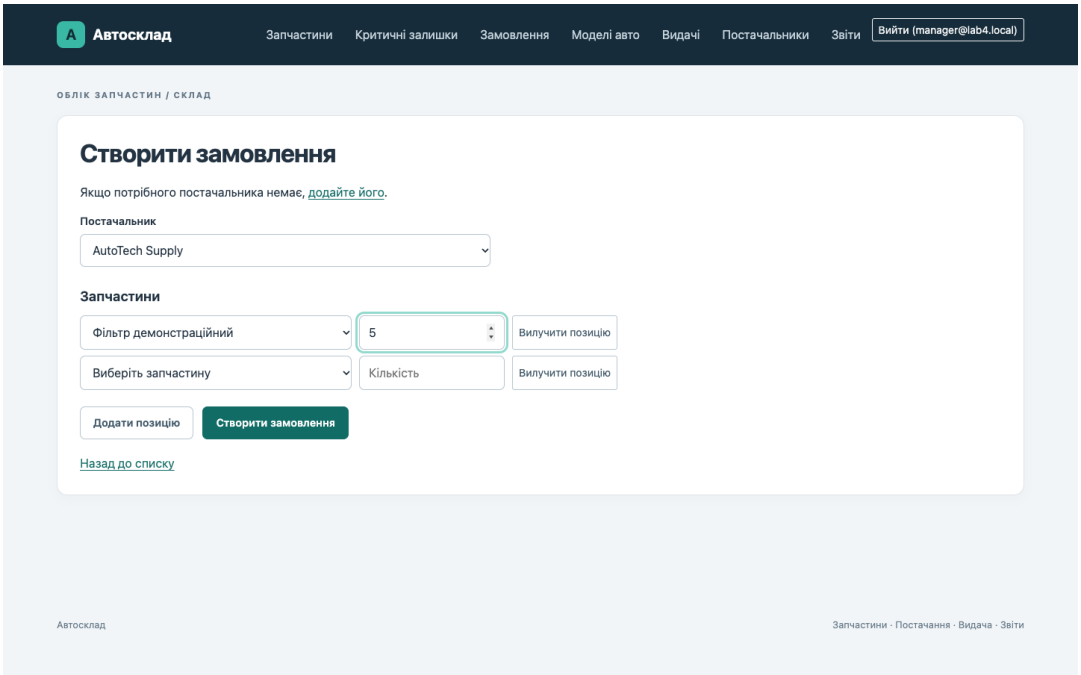


Рисунок 9 – Створення замовлення постачальнику

3.5 Позиції PurchaseOrderItems

PurchaseOrderItem містить Id, PurchaseOrderId, AutoPartId і додатну Quantity. Позиція належить замовленню й посилається на запчастину. Окремого контролера немає: Create та Edit контролера замовлень додають і змінюють позиції через Items. Кнопка «Вилучити позицію» прибирає рядок з форми неотриманого замовлення; після збереження оновлюється набір позицій.

У тесті для однієї запчастини було створено дві позиції, а потім змінено їхні кількості. Receive групує позиції одного AutoPartId, тож до запасу додається сума відповідної запчастини. Для отриманого замовлення редагування позицій заборонене.

3.6 Видачі PartIssues

PartIssue містить Id, AutoPartId, Quantity, CreatedAt, IssuedAt, Status та дані підтвердження оплати, анулювання і повернення. Комірник створює документ методом Issue; усі документи відображає Issues. Менеджер викликає ConfirmPayment, після чого комірник виконує CompleteIssue. Повернення оформлюється методом ReturnIssue, а CancelIssue доступний лише до фактичної видачі.

Фізичне видалення видачі через GUI не реалізоване: стан і часові позначки зберігають історію. Під час тесту Pending не змінив залишку 6, оплачена видача двох одиниць зменшила його до 4, а повне повернення

відновило до 6. Результати окремого демонстраційного сценарію наведено на рисунках у розділі 5.

3.7 Коригування StockAdjustments

StockAdjustment містить Id, AutoPartId, Delta, QuantityBefore, QuantityAfter, Reason, OccurredAt і PerformedByUserId. Комірник створює запис методом AdjustStock; менеджер і комірник переглядають його в Details запчастини. Запис історії не редагується і не видаляється через GUI. Зв’язок із запчастиною та користувачем захищений зовнішніми ключами.

У перевірці для запчастини із залишком 6 введено зміну −2 з причиною «Інвентаризація». Отримано залишок 4 і запис з Delta = −2, QuantityBefore = 6, QuantityAfter = 4. Спробу отримати від’ємний залишок відхилено.

3.8 Користувачі Identity

ApplicationUser розширює обліковий запис Identity полем DisplayName. Identity зберігає користувачів, ролі та службові зв’язки у власних таблицях. AdminController.Index показує користувачів; Create створює обліковий запис і призначає роль; ChangeRole та SetLocked змінюють роль і стан. Фізичного видалення службових таблиць через GUI немає.

На тестовій базі адміністратор створив QA Employee як менеджера, змінив роль на комірника, заблокував і розблокував запис. Після зміни ролі та блокування попередня сесія втрачала доступ; повторний вхід заблокованого користувача не проходив.

4 ПЕРЕВІРКА ВВЕДЕННЯ ТА ЦІЛІСНОСТІ

Обов’язкові поля та діапазони частково задаються атрибутами моделей і ViewModel, а правила, що залежать від інших записів або поточного стану, перевіряють контролери. Якщо ModelState містить помилки, форма повертається з повідомленнями [5]. Таблиця 4 узагальнює перевірки.

Таблиця 4 – Перевірки та обмеження

Сутність	**Перевірка / обмеження**
AutoPart	Назва та виробник обов’язкові; кількості не від’ємні; дублікат назви з виробником відхиляється; історія блокує видалення.
Supplier	Назва й телефон обов’язкові; постачальник із замовленням не видаляється.
VehicleModel	Марка і модель обов’язкові, рік 1886–2100; перевіряється повтор моделі й року.
PurchaseOrder / Item	Постачальник і запчастина мають існувати; потрібна позиція з додатною кількістю; отриманий або анульований документ не редагується.
PartIssue	Кількість додатна; оплата підтверджується до видачі; стан і залишок повторно перевіряються на сервері.

Сутність	Перевірка / обмеження
StockAdjustment	Причина обов’язкова; зміна не дорівнює нулю; новий залишок не може бути від’ємним або перевищувати int.MaxValue.
Identity	Перевіряються пошта, пароль та одна з трьох дозволених ролей; дубль користувача відхиляється.

Спроба повторно додати запчастину «Масляний фільтр» виробника Bosch показала повідомлення про вже наявний запис (рисунок 10). Видалення постачальника із замовленням блокується і контролером, і правилом зовнішнього ключа SQLite. Видалення моделі авто прибирає зв’язки сумісності, але не самі запчастини.

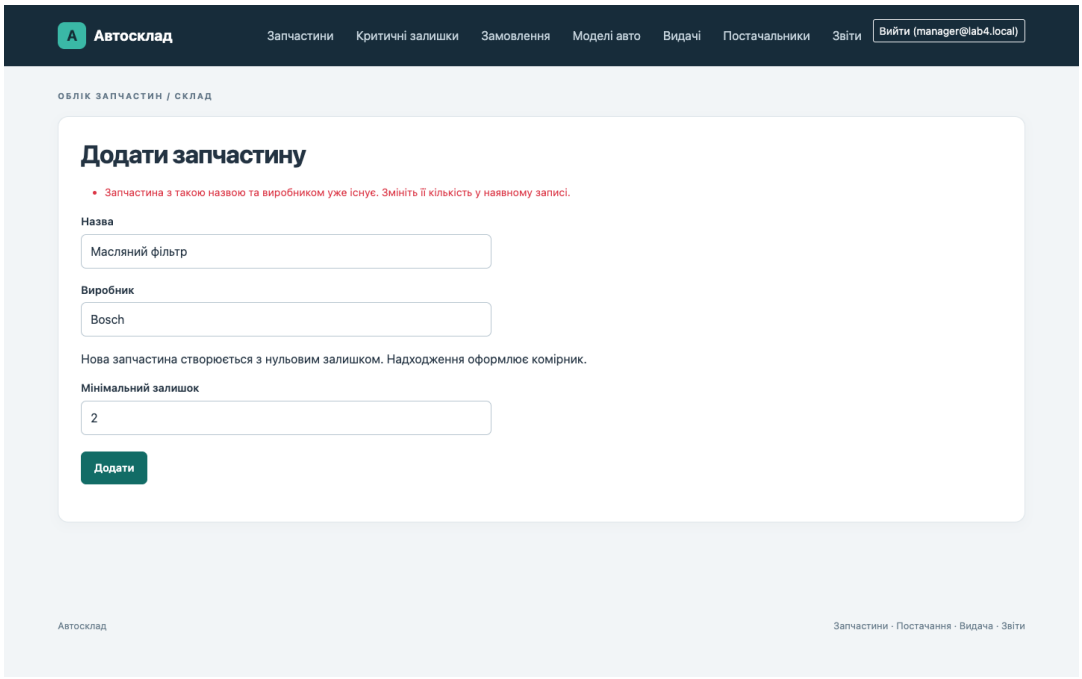


Рисунок 10 – Повідомлення про дублювання запчастини

5 БІЗНЕС-ЛОГІКА СКЛАДСЬКИХ ОПЕРАЦІЙ

Замовлення формується менеджером без зміни складу. Комірник приймає його через Receive: контролер у транзакції завантажує позиції, перевіряє стан та допустимість підсумкового залишку і збільшує кількість за кожною запчастиною. Статус Received перешкоджає повторному прийманню (рисунок 11).

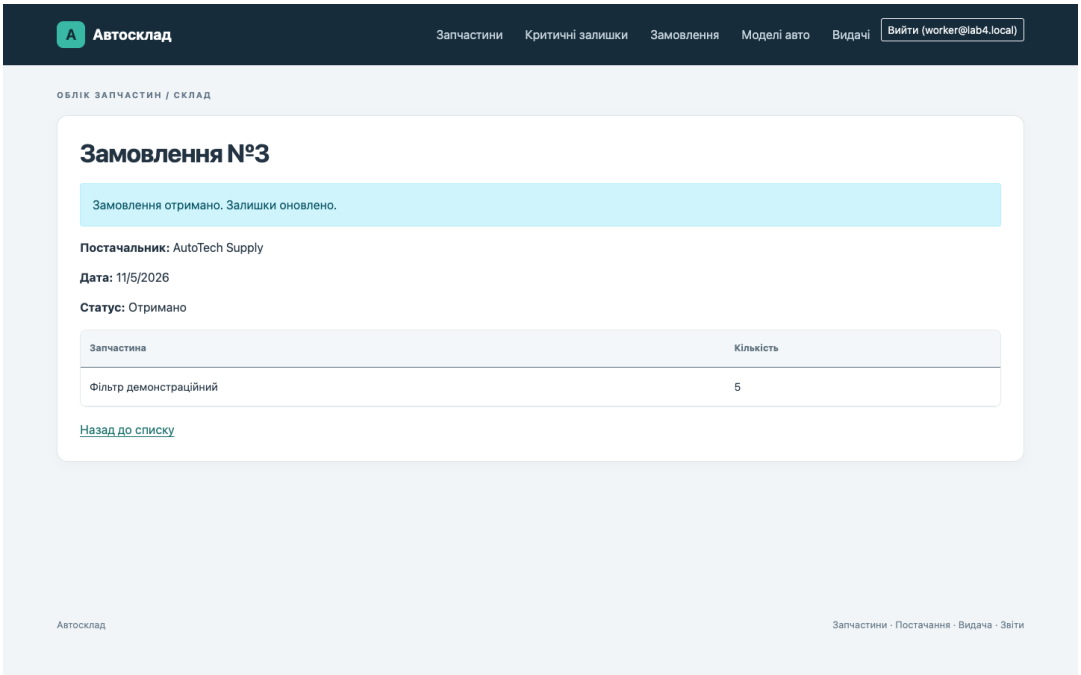


Рисунок 11 – Повідомлення після фізичного приймання поставки

Видача починається зі стану Pending. Issue створює документ з кількістю й автором, не резервуючи і не списуючи запас. Тому перед фізичною видачею залишок перевіряється повторно. Якщо документ помилковий, комірник може анулювати лише Pending, указавши причину; кількість не змінюється.

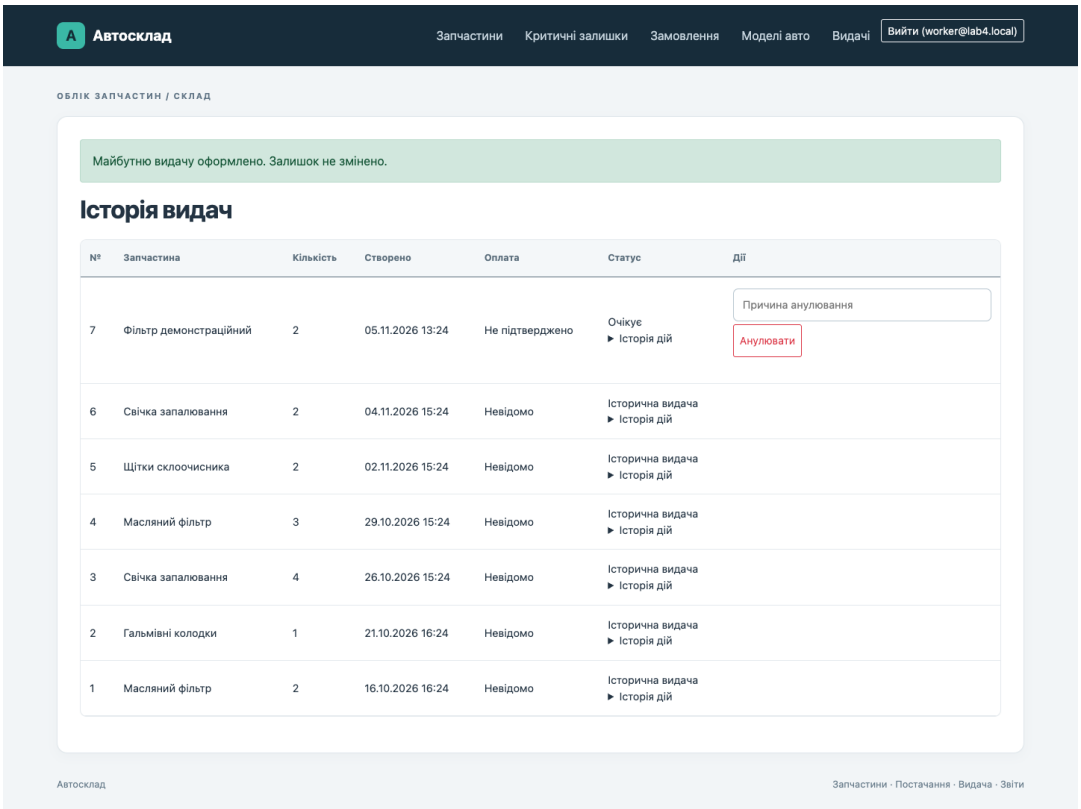


Рисунок 12 – Очікувана видача без зміни складського залишку

Менеджер окремим POST-запитом ConfirmPayment підтверджує оплату Pending-документа. У записі зберігаються прапорець, дата та ідентифікатор менеджера. Це ручне підтвердження, а не інтеграція з платіжною системою. Комірник не має права викликати цю дію.

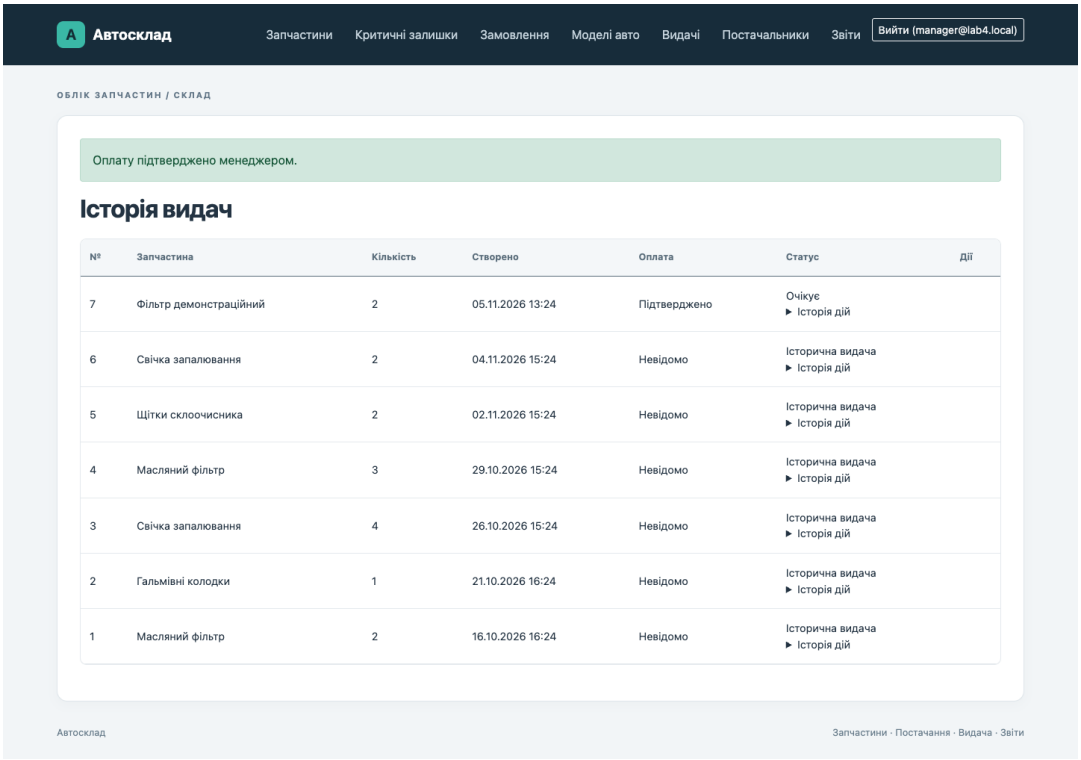


Рисунок 13 – Підтвердження оплати менеджером

Після підтвердження комірник виконує CompleteIssue. Контролер перевіряє Pending, оплату й наявний залишок, після чого в межах транзакції зменшує кількість і встановлює Issued, дату та працівника. Повторна видача й анулювання Issued відхиляються. Нижче наведено фрагмент змін після перевірок.

```
issue.AutoPart.QuantityInStock -= issue.Quantity;
issue.Status = PartIssueStatus.Issued;
issue.IssuedAt = DateTime.UtcNow;
issue.IssuedByUserId = actor;
await _context.SaveChangesAsync();
await transaction.CommitAsync();
```

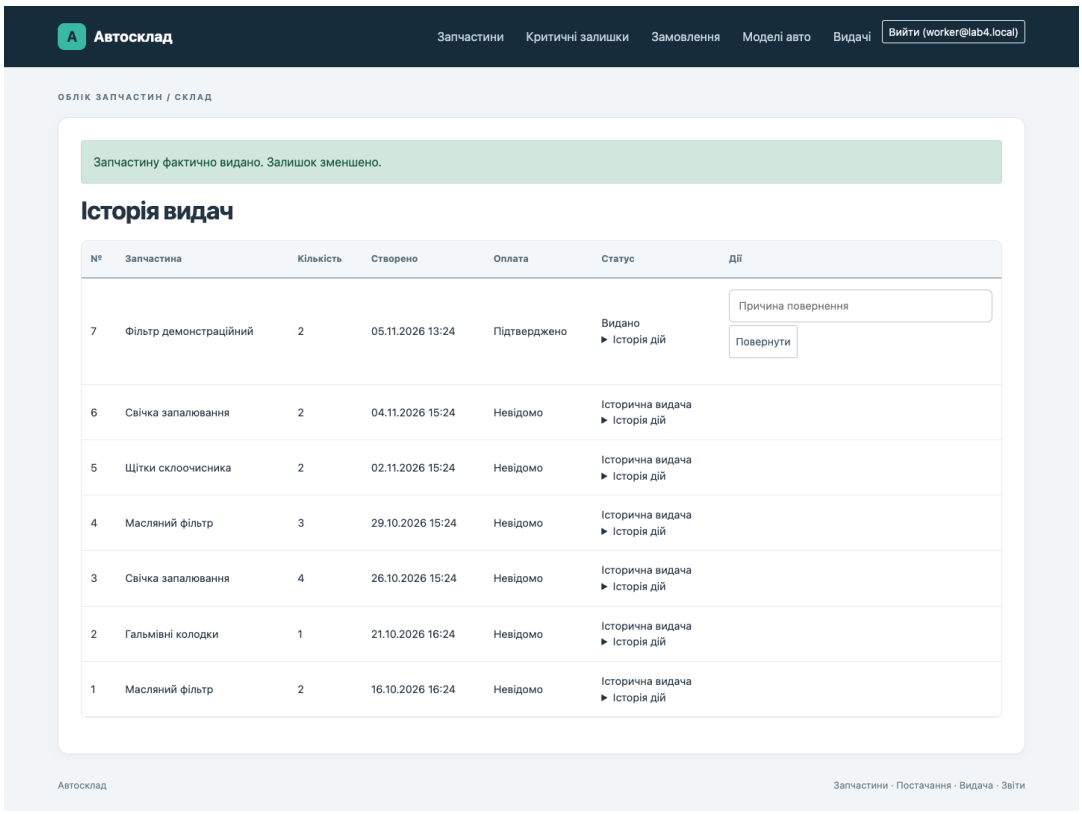


Рисунок 14 – Фактична видача після підтвердження оплати

ReturnIssue допускається тільки зі стану Issued та потребує причини. Повне повернення збільшує залишок, встановлює Returned і зберігає час та комірника. Повторне повернення заборонене. AdjustStock аналогічно записує причину та зміну зі знаком; перевірка нового залишку, запис журналу і зміна кількості виконуються в одній транзакції [6].

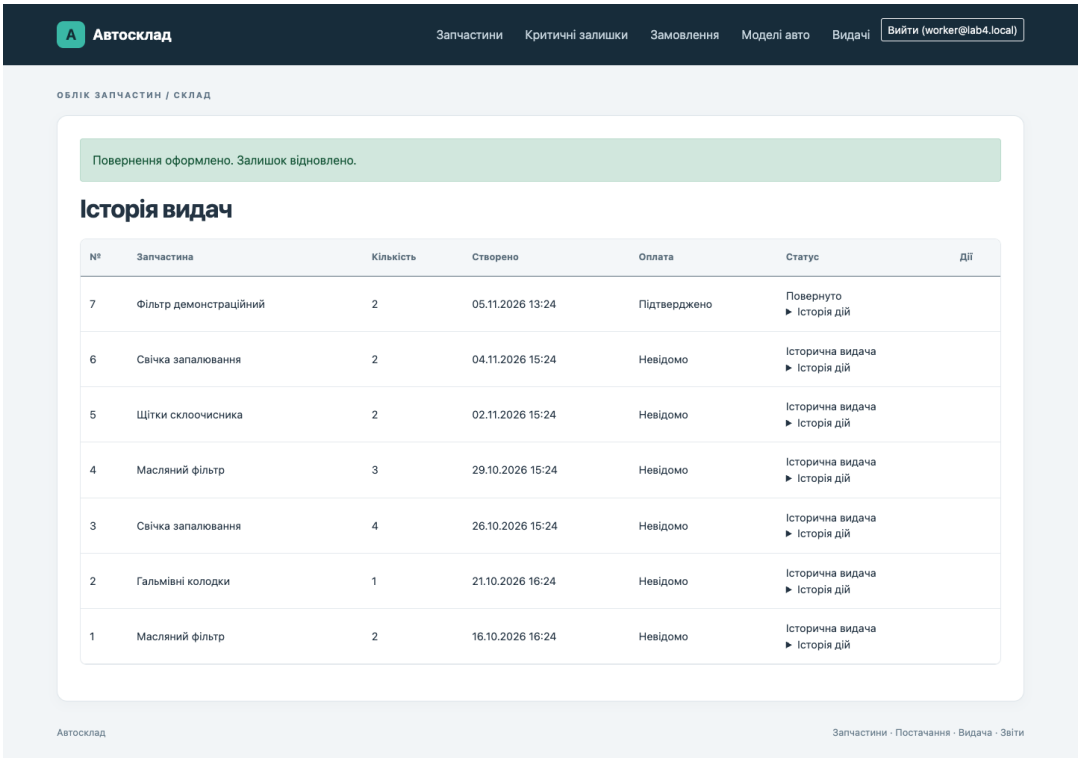


Рисунок 15 – Завершене повне повернення запчастини

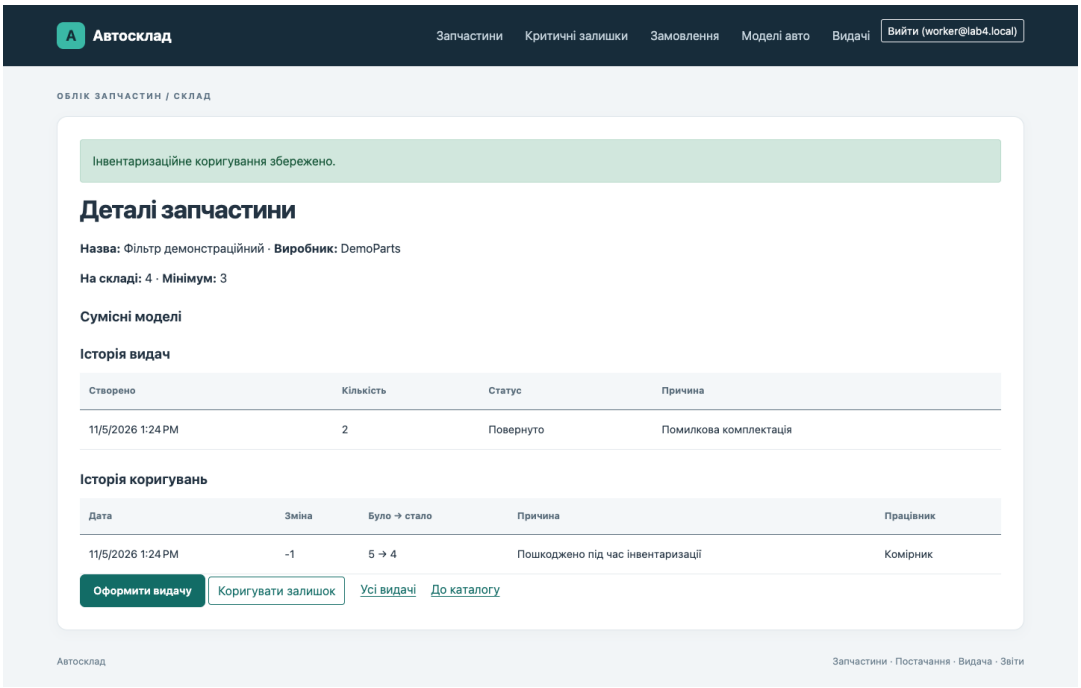


Рисунок 16 – Залишок і журнал інвентаризаційного коригування

Таблиця 5 – Вплив операцій на QuantityInStock

Операція	**Зміна запасу**	**Історія**
Створення замовлення / Pending	0	Створюється документ
Приймання поставки	+ кількість кожної запчастини	Замовлення Received
Підтвердження оплати	0	Дата й менеджер
Фактична видача	− кількість видачі	Issued, дата й комірник
Анулювання Pending	0	Cancelled і причина
Повернення Issued	+ кількість видачі	Returned і причина

Операція	Зміна запасу	Історія
Інвентаризація	Delta зі знаком	До, після, причина і працівник

Раніше створені записи зі старими станами Active та Cancelled переводяться міграцією відповідно в LegacyIssued і LegacyReversed без повторного списання або повернення залишків. Для старих записів відсутність підтвердження оплати не доводить, що оплати не було. Старі скасування позначаються окремо й не входять до агрегату нових повернень.

LINQ-звіт групує дані за конкретною запчастиною. Окремо показано валову видачу за датою видачі, повернення за датою повернення і різницю за вибраний період; історія документів фільтрується за датою створення [7]. Різниця може бути від’ємною, якщо за період повернуто товар, виданий раніше.

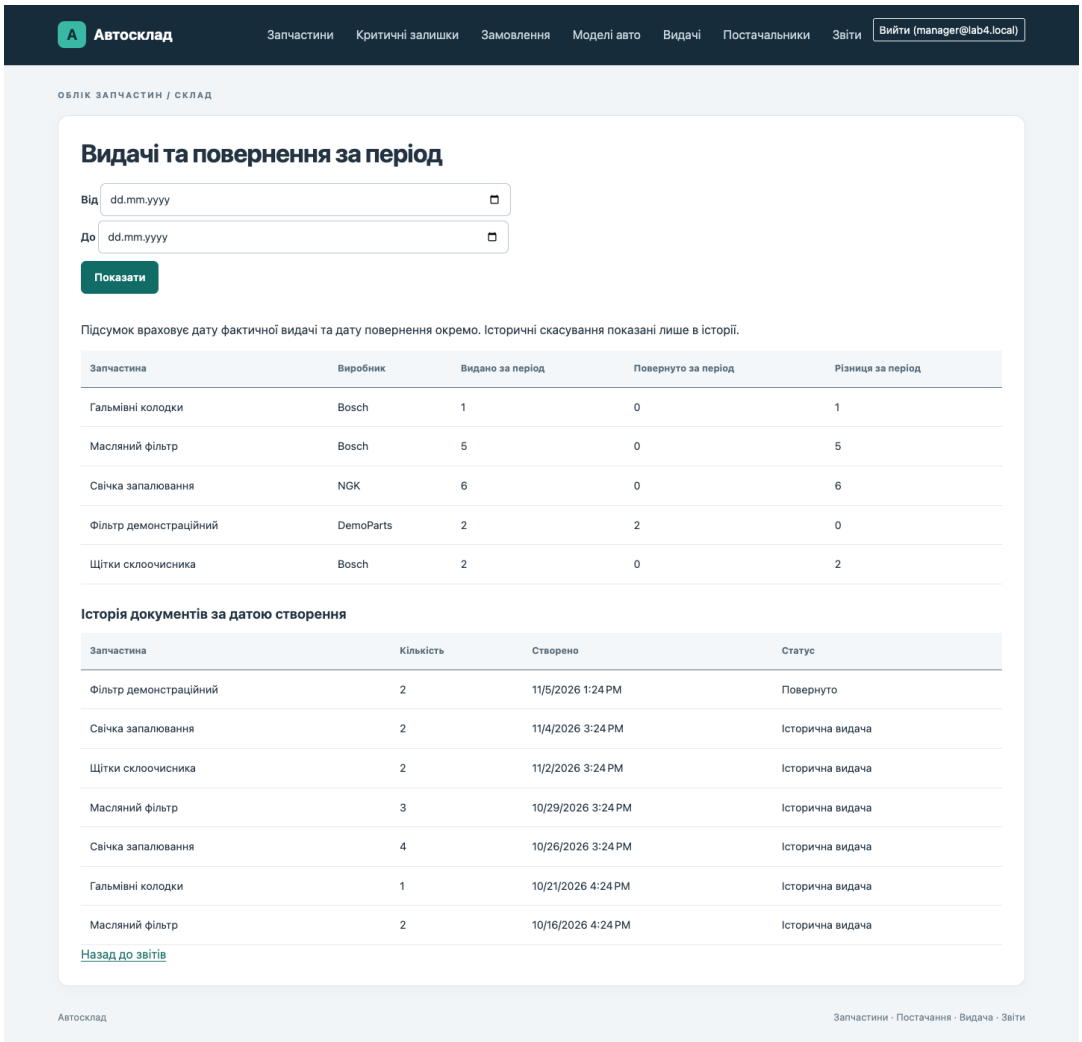


Рисунок 17 – Звіт за запчастинами з видачею, поверненням і різницею

6 ПЕРЕВІРКА РОБОТИ ЗАСТОСУНКУ

Застосунок зібрано командою dotnet build --no-restore: 0 помилок і 0 попереджень. Далі виконано HTTP-перевірки з трьома окремими сесіями та контрольними запитами SQLite на чистій тестовій базі. Браузерні сценарії виконано в Chrome на macOS; робоча база не використовувалася для цих операцій.

Для основного сценарію автоматизованої перевірки створено QA Test Part з початковим залишком 0. Дві позиції замовлення після редагування містили 4 і 2 одиниці. Отримання дало 6, видача двох одиниць – 4, повернення – 6, коригування −2 – 4. Очікувані та фактичні результати наведено в таблиці 6.

Таблиця 6 – Сценарії перевірки

№	Дія	Очікуваний результат	Фактичний результат
1	Вхід адміністратора	Відкрити користувачів	Вхід успішний; сторінка Admin
2	Вхід менеджера і комірника	Відповідна навігація	Обидва входи успішні
3	Анонімний доступ до каталогу	Перенаправлення на вхід	Отримано перенаправлення
4	Комірник викликає ConfirmPayment	Доступ заборонено	Сервер відмовив у доступі
5	Менеджер викликає Receive	Доступ заборонено	Сервер відмовив у доступі
6	Створити запчастину з переданою кількістю 999	Початковий залишок 0	Записано 0
7	Редагувати характеристики з полем кількості 999	Кількість не змінюється	Назву змінено, залишок 0
8	Повторити назву та виробника	Дублікат не додається	Залишився один запис
9	Порожні поля та від’ємне число	Помилка перевірки	Новий запис не створено
10	Створити та відредагувати постачальника	Дані зберігаються	Створення та зміна виконані
11	Видалити постачальника з замовленням	Вилучення заборонено	Запис залишився, показано помилку
12	Створити й змінити сумісність моделі	Зберегти вибрані зв’язки	Два зв’язки змінено на один
13	Видалити модель авто	Запчастина залишається	Модель видалена, запчастина є
14	Створити замовлення з двома позиціями	Залишок не змінюється	Залишок 0
15	Прийняти 4 + 2 одиниці	Залишок 6	Отримано 6; стан Received
16	Повторити приймання	Друга зміна заборонена	Залишок залишився 6
17	Створити Pending на 2 одиниці	Залишок не змінюється	Pending; залишок 6
18	Видати без оплати	Операція відхиляється	Статус лишився Pending
19	Підтвердити оплату і видати	Залишок 4, Issued	Залишок 4; дата й працівник збережені

№	Дія	Очікуваний результат	Фактичний результат
20	Повторити видачу / анулювати Issued	Операції відхиляються	Залишок 4; стан Issued
21	Повернути дві одиниці і повторити	Залишок 6 лише один раз	Returned; залишок 6
22	Анулювати інший Pending з причиною	Залишок без зміни	Cancelled; залишок 6
23	Коригування −2 та спроба −20	4; від’ємний результат заборонено	Записано −2; залишок 4
24	Звіт після видачі та повернення	Видано 2, повернуто 2, різниця 0	Виведено 2 / 2 / 0
25	Два одночасні запити на останню одиницю	Видати один раз	Одна видача; залишок 0
26	Подвійне одночасне повернення / приймання	Одна зміна залишку	Кожну операцію виконано один раз

Додатково перевірено створення облікового запису, зміну ролі, блокування й розблокування та захист POST без antiforgery-токена. Міграцію перевірено на копії бази: сім старих записів видач збережено, залишки не змінилися, integrity_check і foreign_key_check не виявили порушень. Сценарії одночасних запитів перевірено для двох паралельних звернень, а не як навантажувальне випробування.

ВИСНОВКИ

У лабораторній роботі доповнено інтерфейс системи складу автозапчастин засобами керування довідниками, замовленнями та складськими операціями. Реалізація на C# з ASP.NET Core MVC відокремлює моделі, контролери й Razor-подання; Entity Framework Core працює з SQLite.

Для довідників доступні операції CRUD із перевірками зв’язків. Для замовлень і видач використано контрольовані стани замість небезпечного фізичного видалення. Видача і повернення змінюють залишок у транзакції, а інвентаризаційне коригування зберігає причину й автора. Доступ розмежовано між адміністратором, менеджером і комірником.

Перевірки підтвердили основні позитивні та негативні сценарії на ізольованій базі. Таким чином, створено графічний інтерфейс і реалізовано роботу із записами бази відповідно до завдань лабораторної роботи, з урахуванням цілісності складської історії.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Огляд ASP.NET Core MVC. Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/mvc/overview?view=aspnetcore-8.0 (дата звернення: 05.11.2026).
2. Огляд міграцій Entity Framework Core. Microsoft Learn. URL: https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/ (дата звернення: 05.11.2026).
3. Провайдер SQLite для Entity Framework Core. Microsoft Learn. URL: https://learn.microsoft.com/en-us/ef/core/providers/sqlite/ (дата звернення: 05.11.2026).
4. Авторизація на основі ролей в ASP.NET Core. Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/security/authorization/roles?view=aspnetcore-8.0 (дата звернення: 05.11.2026).
5. Перевірка моделей в ASP.NET Core MVC. Microsoft Learn. URL: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation?view=aspnetcore-8.0 (дата звернення: 05.11.2026).
6. Транзакції в Entity Framework Core. Microsoft Learn. URL: https://learn.microsoft.com/en-us/ef/core/saving/transactions (дата звернення: 05.11.2026).
7. Вступ до LINQ у C#. Microsoft Learn. URL: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/introduction-to-linq (дата звернення: 05.11.2026).